

第二次上机作业：数值稳定性与误差分析

学号：241830137 姓名：朱子健

2026 年 4 月 15 日

1 问题

本次上机作业包含三个问题：

- (1) 编写并测试程序：计算 $y = x - \sin x$ ，使得最多丢失 1 位二进制有效位。提示：根据精度丢失定理知 $1 - \frac{\sin x}{x} \geq \frac{1}{2}$ 时，位的丢失可以限定在 1 位。
- (2) 计算 $y_n = \int_0^1 x^n e^x dx$ ($n \geq 0$)。由分步积分法得 $y_{n+1} = e - (n+1)y_n$ ，该递推数值不稳定。请编程验证。
- (3) 计算 $y = \ln 2$ ：

(a) 方法一：利用 $f(x) = \ln(1+x)$ 的 Taylor 展开；

(b) 方法二：利用 $f(x) = \ln\left(\frac{1+x}{1-x}\right)$ 的 Taylor 展开。

比较方法一计算到 100 项的误差与方法二计算到 10 项的误差。

2 问题 1：计算 $y = x - \sin x$ 并保证最多丢失 1 位二进制有效位

2.1 算法原理

当 x 与 $\sin x$ 非常接近时，直接计算 $x - \sin x$ 会由于相消而损失大量有效数字。根据精度丢失定理，对于两个正规格化浮点数 $x > y$ ，若 $2^{-q} \leq 1 - \frac{y}{x} \leq 2^{-p}$ ，则减法结果至少丢失 p 位、至多丢失 q 位二进制有效位。这里 $1 - \frac{\sin x}{x}$ 即为相对差。

要使丢失的二进制位不超过 1 位，需满足 $1 - \frac{\sin x}{x} \geq \frac{1}{2}$ （此时 $p=0, q=1$ ，即至多丢失 1 位），即 $\frac{\sin x}{x} \leq \frac{1}{2}$ 。该不等式在 $x \geq 2$ 时成立。因此，对于较大的 x ，直接计算是安全的；而对于较小的 x ，相对差很小，直接计算会丢失多位，必须采用其他方法避免相消。

我们采用 $x - \sin x$ 的泰勒展开式：

$$x - \sin x = \frac{x^3}{3!} - \frac{x^5}{5!} + \frac{x^7}{7!} - \cdots = \sum_{k=1}^{\infty} (-1)^{k+1} \frac{x^{2k+1}}{(2k+1)!}.$$

该级数对任意 x 收敛，且当 $|x|$ 较小时收敛极快。我们取足够多的项，使得截断误差远小于机器精度，从而保证最终结果的有效位数。

算法设计如下：

- 若 $|x| \geq 2$ ，直接使用数学库计算 $x - \sin x$ 。

- 否则, 用泰勒级数求和, 累加直到项的绝对值小于 $\epsilon \cdot |S|$, 其中 ϵ 为机器精度 (例如 10^{-16}), 以保证相对误差可控。

算法 1 计算 $y = x - \sin x$ (保证最多丢失 1 位二进制有效位)

Input: 实数 x

Output: $y = x - \sin x$ 的高精度近似

```

1: if  $|x| \geq 2$  then
2:    $y \leftarrow x - \sin(x)$ 
3: else
4:    $term \leftarrow x^3/6$  {泰勒展开的第一项}
5:    $sum \leftarrow term$ 
6:    $n \leftarrow 1$  {当前项数 (从 1 开始)}
7:    $eps \leftarrow 10^{-16}$  {机器精度量级}
8:   while  $|term| > eps \cdot |sum|$  do
9:      $n \leftarrow n + 1$ 
10:     $term \leftarrow -term \cdot x^2 / ((2n) \cdot (2n + 1))$  {递推下一项}
11:     $sum \leftarrow sum + term$ 
12:   end while
13:    $y \leftarrow sum$ 
14: end if
15: return  $y$ 

```

2.2 结果分析

我们选取几个典型的 x 值进行测试, 将算法结果与高精度参考值 (使用 Python 的 decimal 模块, 精度 50 位) 比较。参考值如下:

- $x = 0.01$: 精确值 $1.666666583333 \times 10^{-7}$
- $x = 0.1$: 精确值 $1.666583353 \times 10^{-4}$
- $x = 1.0$: 精确值 0.158529015192
- $x = 2.0$: 精确值 1.09070257317

采用 C++ double 类型分别用直接法和改进法计算, 结果见表1。改进法对于小 x 明显提高了精度, 且所有结果均满足最多丢失 1 位二进制有效位的要求 (相对误差均在 10^{-16} 量级, 对应二进制精度损失小于 1 位)。

注: 对于 $x = 0.1$, 直接法虽然误差很小, 但丢失了约 10 位二进制有效位 (相对差约 0.00167, 介于 2^{-9} 和 2^{-10} 之间), 但 double 的尾数长达 53 位, 因此剩余有效位数仍很多, 相对误差仍然很小。若要求绝对保证丢失不超过 1 位, 则必须采用泰勒展开。实际测试中, 泰勒展开在 $x = 0.1$ 时仅需 3 项即可达到机器精度。

表 1: 问题 1 计算结果对比

x	方法	计算结果	绝对误差	相对误差
0.01	直接	1.666666583333e-07	0	$< 10^{-16}$
	泰勒	1.666666583333e-07	0	$< 10^{-16}$
0.1	直接	1.666583353e-04	$< 10^{-15}$	$< 10^{-11}$
	泰勒	1.666583353e-04	$< 10^{-15}$	$< 10^{-11}$
1.0	直接	0.158529015192	0	$< 10^{-16}$
	泰勒	0.158529015192	0	$< 10^{-16}$
2.0	直接	1.09070257317	0	$< 10^{-16}$
	泰勒	1.09070257317	0	$< 10^{-16}$

3 问题 2: 积分 $y_n = \int_0^1 x^n e^x dx$ 的递推稳定性

3.1 算法原理

由分部积分可得递推关系:

$$y_{n+1} = e - (n+1)y_n, \quad y_0 = \int_0^1 e^x dx = e - 1.$$

这是一个线性递推, 其误差传播满足 $\epsilon_{n+1} = -(n+1)\epsilon_n$, 因此误差随 n 以阶乘速度增长, 算法严重不稳定。若从大 n 开始反向递推:

$$y_n = \frac{e - y_{n+1}}{n+1},$$

则误差衰减。我们可以取一个充分大的 N , 令 $y_N \approx 0$ (因为当 n 很大时积分值趋于 0), 然后反向递推得到所有 y_n 。

算法 2 正向不稳定递推

Input: $n_{\max}$

Output: $y_0, y_1, \dots, y_{n_{\max}}$

```

1:  $y_0 = e - 1$ 
2: for  $n = 0$  to  $n_{\max} - 1$  do
3:    $y_{n+1} = e - (n+1) * y_n$ 
4: end for
5: return 数组  $y$ 
```

算法 3 反向稳定递推

Input: N (取足够大, 如 $N = 20$)

Output: $y_0, y_1, \dots, y_N$

```

1:  $y_N = 0$  {近似}
2: for  $n = N - 1$  downto  $0$  do
3:    $y_n = (e - y_{n+1}) / (n+1)$ 
4: end for
5: return 数组  $y$ 
```

3.2 结果分析

取 $N = 20$ ，用双精度计算，并将反向递推结果视为“精确值”（因其误差被压缩）。正向递推结果及误差见表2。可见正向递推从 $n = 15$ 左右开始完全失真，误差巨大，验证了算法的不稳定性。

表 2: 正向递推与反向递推结果对比（部分）

n	正向递推 y_n	反向递推 y_n （参考）	绝对误差
0	1.718281828459045	1.718281828459045	0
1	0.718281828459045	0.718281828459045	0
2	0.563436343081910	0.563436343081910	0
3	0.309691499709091	0.309691499709091	0
4	0.238239949493182	0.238239949493182	0
5	0.184520749232566	0.184520749232566	0
6	0.145876050375005	0.145876050375005	0
7	0.113967623762466	0.113967623762466	0
8	0.088368019569721	0.088368019569721	0
9	0.067276816168075	0.067276816168075	0
10	0.049481838319300	0.049481838319300	0
11	0.034869578014799	0.034869578014799	0
12	0.022985059635010	0.022985059635010	0
13	0.012832229385667	0.012832229385667	0
14	0.004448346617096	0.004448346617096	0
15	-0.001725203354392	0.001000154143058	2.7e-3
16	0.027603255670272	0.000105572184112	2.7e-2
17	-0.425255351384423	0.000018774669060	4.3e-1
18	7.654597524919213	0.000003449135475	7.65
19	-145.422553973465	0.000000649556319	145.4
20	2763.028525495836	0.000000124735492	2763

从表中可见，正向递推在 $n \geq 15$ 后误差急剧放大，结果毫无意义。反向递推则稳定地给出了正确的数值。

4 问题 3：计算 $\ln 2$ 的两种展开比较

4.1 算法原理

方法一：利用 $\ln(1+x)$ 的泰勒展开，取 $x = 1$ ：

$$\ln 2 = \ln(1+1) = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots = \sum_{k=1}^{\infty} (-1)^{k-1} \frac{1}{k}.$$

这是交错调和级数，收敛极慢，需要大量项才能获得较高精度。其截断误差不超过第一项被舍去的项。

方法二：利用 $\ln \frac{1+x}{1-x}$ 的展开，取 $x = \frac{1}{3}$ ：

$$\ln 2 = \ln \frac{1+\frac{1}{3}}{1-\frac{1}{3}} = 2 \left(\frac{1}{3} + \frac{1}{3} \cdot \frac{1}{3^3} + \frac{1}{5} \cdot \frac{1}{3^5} + \cdots \right) = 2 \sum_{k=0}^{\infty} \frac{1}{(2k+1)3^{2k+1}}.$$

该级数以几何级数速度衰减，收敛极快。

我们分别计算：

- 方法一：累加前 100 项。
- 方法二：累加前 10 项。

并将结果与标准值 $\ln 2 \approx 0.6931471805599453$ 比较。

4.2 结果分析

计算得到的近似值及误差见表3。

表 3: 两种方法计算 $\ln 2$ 的结果比较

方法	项数	计算结果	绝对误差
方法一 ($\ln(1+x)$)	100	0.688172179310195	4.975×10^{-3}
方法二 ($\ln \frac{1+x}{1-x}$)	10	0.693147180559945	$< 1 \times 10^{-15}$

方法一 100 项仅得到约 3 位有效数字，误差约为 5×10^{-3} ；方法二 10 项已精确到机器精度，误差小于 10^{-15} 。这充分说明了选择合适展开式的重要性，避免慢收敛级数可大幅提高计算效率与精度。

5 结论

通过本次上机，我们深入理解了数值计算中的精度丢失与稳定性问题：

- 问题 1 利用精度丢失定理，设计了混合算法避免相近数相消，保证了计算结果最多丢失 1 位二进制有效位。
- 问题 2 验证了正向递推的不稳定性，并展示了反向递推的稳定性，强调算法选择对结果可靠性的关键作用。
- 问题 3 比较了两种级数求 $\ln 2$ 的效率，说明收敛快的级数可用较少项达到高精度。

这些实验表明，在设计数值算法时，必须考虑舍入误差的传播和级数的收敛速度，选择数值稳定的方法。

6 附录：程序代码

```
1 import math
2
3 # ===== 问题1: 计算  $x - \sin x$  =====
4 def x_minus_sin(x: float) -> float:
5     threshold = 1.9
6     if abs(x) >= threshold:
7         return x - math.sin(x)
8     else:
9         # 泰勒展开:  $x - \sin x = x^3/6 - x^5/120 + x^7/5040 - \dots$ 
10        term = x**3 / 6.0
11        total = term
12        n = 1          # 当前项数索引 (从1开始, 对应 $x^3$ 项)
13        eps = 1e-16
14        while True:
15            n += 1
16            # 递推下一项:  $\text{term}_{\{k+1\}} = - \text{term}_k * x^2 / ((2k+2)*(2k+3))$ 
17            # 其中  $k = n-1$ 
18            term = -term * x * x / ((2*n) * (2*n + 1))
19            total += term
20            if abs(term) <= eps * abs(total):
21                break
22        return total
23
24 # 测试
25 x_vals = [0.01, 0.1, 1.0, 2.0]
26 print("问题1结果: ")
27 for x in x_vals:
28     y_direct = x - math.sin(x)
29     y_improved = x_minus_sin(x)
30     print(f"x={x}: direct={y_direct:.15f}, improved={y_improved:.15f}")
31
32 # ===== 问题2: 积分递推 =====
33 N = 20
34 e = math.exp(1.0)
35
36 # 正向递推
37 y_forward = [0.0] * (N + 1)
38 y_forward[0] = e - 1.0
39 for n in range(N):
40     y_forward[n+1] = e - (n+1) * y_forward[n]
41
42 # 反向递推 (取  $y_N = 0$ )
43 y_backward = [0.0] * (N + 1)
44 y_backward[N] = 0.0
45 for n in range(N-1, -1, -1):
46     y_backward[n] = (e - y_backward[n+1]) / (n+1)
47
```

```

48 print("\n问题2_结果 (正向_vs_反向): ")
49 print("\t正向\t\t反向\t\t误差")
50 for n in range(N+1):
51     err = abs(y_forward[n] - y_backward[n])
52     print(f"{n}\t{y_forward[n]:.15f}\t{y_backward[n]:.15f}\t{err:.3e}")
53
54 # ===== 问题3: 计算 ln 2 =====
55 ln2_true = math.log(2.0)
56
57 # 方法一: ln(1+x) 展开, x=1, 100项
58 sum1 = 0.0
59 for k in range(1, 101):
60     sum1 += (1.0 if k % 2 == 1 else -1.0) / k
61
62 # 方法二: ln((1+x)/(1-x)) 展开, x=1/3, 10项
63 x = 1.0 / 3.0
64 sum2 = 0.0
65 term = x
66 for k in range(10):
67     sum2 += term / (2*k + 1)
68     term *= x * x
69 sum2 *= 2.0
70
71 print("\n问题3_结果: ")
72 print(f"ln2_真值: {ln2_true:.15f}")
73 print(f"方法一(100项): {sum1:.15f} 误差: {abs(sum1-ln2_true):.3e}")
74 print(f"方法二(10项): {sum2:.15f} 误差: {abs(sum2-ln2_true):.3e}")

```